

Как да напишем `cells.cpp`

ръководство стъпка по стъпка

Microsoft Visual Studio · C++17 · try/catch, typecast, template

Какво ще направим

Ще опишем **клетка на таблица** — като в електронна таблица. Една и съща клетка може да съдържа число или текст, а програмата трябва да ги обработва различно, без да знае предварително кое къде е.

Идеята е една: стойността винаги се **пази** като низ (`string m_str_val`), а **тълкуването** ѝ зависи от вида на клетката. Оттам идват и трите нови теми в упражнението.

Тема	Къде се появява
Типово преобразуване	Низът <code>"42"</code> трябва да стане числото <code>42</code> . А какво става с <code>"не-число"</code> ?
Изключения	Точно затова. Невалидна стойност или невалиден индекс не могат да се върнат като резултат — трябва механизъм, който да прекъсне работата и да съобщи какво е станало.
Шаблони	Класът <code>CLess</code> сравнява <i>каквото и да било</i> , стига да има <code>operator<</code> — един код за <code>int</code> , <code>string</code> и за нашите класове.

План на работата

1. Нов проект и скелет
2. Собствен клас за изключения `CCellException`
3. Абстрактният клас `CCellDescrBase` (изискване I)
4. Мутатори с проверка — първите изключения
5. `CCellDescrInteger` и типовото преобразуване (изискване II)
6. Операторите на числовата клетка (изискване II)
7. `CCellDescrString` (изискване III)
8. Шаблонният клас `CLess`
9. Главна функция (изискване IV)

1 Нов проект и скелет

Нов **Empty Project**, например `Tema6_Cells`, файл `cells.cpp`.

```
#include <cstdio>      // getchar
#include <exception>
#include <iomanip>      // setw, left, right
#include <iostream>
#include <stdexcept>    // invalid_argument, out_of_range
```

```
#include <string>
using namespace std;
```

2 Собствен клас за изключения

Изключение може да бъде всякакъв тип — дори `int`. Но тогава уловилят го няма как да разбере какво се е случило. Затова се прави клас, наследник на `std::exception`.

ДОБАВЕТЕ НАЙ-ОТГОРЕ

```
class CCellException : public exception
{
private:
    string m_strMsg;

public:
    explicit CCellException(const string& strMsg) : m_strMsg(strMsg) {}

    // what() е виртуална в std::exception – подменяме я.
    // noexcept е задължително: базовата версия обещава да не хвърля.
    virtual const char* what() const noexcept { return m_strMsg.c_str(); }
};
```

- 1 **Наследник на `std::exception`** — така един `catch (const exception&)` улавя и нашите, и стандартните изключения.
- 2 **`what()` е виртуална.** Подменяме я, за да върне нашето съобщение. Връща `const char*`, не `string` — затова `.c_str()`.
- 3 **`noexcept` е задължително.** Ако го пропуснете, сигнатурата се разминава с базовата и компилаторът отказва: `C2694: overriding virtual function has less restrictive exception specification`.
- 4 **`explicit`** пред конструктора пречи низ случайно да се преобразува в изключение при подаване на параметър.

Защо съобщението се пази в член-променлива? Защото изключението се хвърля от една функция и се улавя в друга — дотогава локалните променливи на хвърлящата функция вече не съществуват. Обектът-изключение се копира и пътува нагоре сам със себе си.

3 Абстрактният клас `CCellDescrBase`

ДОБАВЕТЕ ПОД КЛАСА ЗА ИЗКЛЮЧЕНИЯ

```
class CCellDescrBase
{
// Скрити член променливи. protected, за да ги достигат наследниците.
protected:
    string m_str_val;    // стойност на клетката, представя цяло число или низ
    int    m_row;       // ред
    int    m_col;       // колона
};
```

```

public:
    // Границите на таблицата – статични, общи за всички клетки
    static const int MAX_ROW = 100;
    static const int MAX_COL = 26;

    CCellDescrBase(); // подразбиращ се
    CCellDescrBase(const string& strVal, int nRow, int nCol); // експлицитен
    CCellDescrBase(const CCellDescrBase& cell); // копиращ

    virtual ~CCellDescrBase(); // виртуален!

    int getRow() const;
    int getCol() const;
    void setRow(int nRow); // хвърля CCellException при невалиден ред
    void setCol(int nCol); // хвърля CCellException при невалидна колона

    const string& getValue() const;
    void setValue(const string& strVal);

    string getAddress() const; // A1, B3, ...

    // Напълно виртуална функция за извеждане в поток
    virtual ostream& Output(ostream& toStream) const = 0;
};

const int CCellDescrBase::MAX_ROW;
const int CCellDescrBase::MAX_COL;

```

Какво е static const int в класа? Константа, обща за всички обекти — не се пази отделно във всяка клетка. Пише се `CCellDescrBase::MAX_ROW`. Двата реда след класа я „дефинират“; при C++17 те не са задължителни, но не пречат и правят кода съвместим с по-стари компилатори.

КОНСТРУКТОРИТЕ

```

CCellDescrBase::CCellDescrBase()
    : m_str_val(""), m_row(0), m_col(0) {}

CCellDescrBase::CCellDescrBase(const string& strVal, int nRow, int nCol)
    : m_str_val(strVal), m_row(0), m_col(0)
{
    setRow(nRow); // през мутаторите, за да мине проверката
    setCol(nCol);
}

CCellDescrBase::CCellDescrBase(const CCellDescrBase& cell)
    : m_str_val(cell.m_str_val), m_row(cell.m_row), m_col(cell.m_col) {}

CCellDescrBase::~CCellDescrBase() {}

```

Обърнете внимание на експлицитния конструктор. Той *не* записва `nRow` направо в списъка за инициализация, а вика `setRow(nRow)` в тялото. Причината: проверката за валидност е в мутатора и трябва да важи и при създаване. Ако беше `m_row(nRow)`, клетка с ред `-5` щеше да се създаде безпроблемно. Затова в списъка първо се слага `m_row(0)` — членът трябва да има стойност, преди тялото да започне.

АДРЕСЪТ НА КЛЕТКАТА

```
string CCellDescrBase::getAddress() const
{
    string strAddr;
    strAddr += static_cast<char>('A' + m_col);
    strAddr += to_string(m_row + 1);
    return strAddr;
}
```

'A' + m_col е обикновена аритметика върху символи: 'A' + 0 е 'A', 'A' + 1 е 'B'. Резултатът е int, затова static_cast<char> го връща обратно към символ. m_row + 1, защото за потребителя редовете започват от 1, а вътрешно — от 0.

ЕДИН ОПЕРАТОР ЗА ЦЯЛАТА ЙЕРАРХИЯ

```
ostream& operator<<(ostream& toStream, const CCellDescrBase& cell)
{
    return cell.Output(toStream);
}
```

Същата постановка като в упражнение 5: глобалният оператор делегира на виртуалната Output. Затова cout << *arrCells[i] извежда числата по един начин, а текста — по друг, без нито едно if.

4 Мутатори с проверка — първите изключения

```
void CCellDescrBase::setRow(int nRow)
{
    if (nRow < 0 || nRow >= MAX_ROW)
        throw CCellException("Невалиден ред: " + to_string(nRow) +
                               " (допустимо 0.." + to_string(MAX_ROW - 1) + ")");
    m_row = nRow;
}

void CCellDescrBase::setCol(int nCol)
{
    if (nCol < 0 || nCol >= MAX_COL)
        throw CCellException("Невалидна колона: " + to_string(nCol) +
                               " (допустимо 0.." + to_string(MAX_COL - 1) + ")");
    m_col = nCol;
}
```

ЗАЩО ИЗКЛЮЧЕНИЕ, А НЕ ВРЪЩАНЕ НА FALSE

Подход	Проблем
bool setRow(int)	Извикващият може да пренебрегне резултата и да не забележи нищо. Освен това конструкторът не може да върне стойност — а точно там проверката е най-нужна.
изход от програмата	Отнема на извикващия всякакъв избор.

`throw`

Не може да бъде пренебрегнато. Ако никой не го улови, програмата спира с ясно съобщение. Работи и в конструктор.

Какво прави `throw`. Изпълнението на функцията спира незабавно — редът `m_row = nRow;` не се стига. Управлението се връща нагоре през извикващите функции, докато не срещне `catch`, който приема този тип. По пътя всички локални обекти се унищожават правилно (викат се деструкторите им) — това се нарича *развиване на стека*.

Ако никъде няма подходящ `catch`, се вика `terminate()` и програмата приключва аварийно.

5 CCellDescrInteger и типовото преобразуване

КЛАСЪТ

```
class CCellDescrInteger : public CCellDescrBase
{
public:
    CCellDescrInteger();
    CCellDescrInteger(const string& strVal, int nRow, int nCol);
    CCellDescrInteger(int nVal, int nRow, int nCol);          // удобен вариант
    CCellDescrInteger(const CCellDescrInteger& cell);
    virtual ~CCellDescrInteger();

    int getInt() const;                                     // може да хвърли

    CCellDescrInteger& operator=(const CCellDescrInteger& cell);
    bool operator==(const CCellDescrInteger& cell) const;
    bool operator< (const CCellDescrInteger& cell) const;

    virtual ostream& Output(ostream& toStream) const;
};
```

Конструкторите просто препращат на базовия:

```
CCellDescrInteger::CCellDescrInteger() : CCellDescrBase("0", 0, 0) {}

CCellDescrInteger::CCellDescrInteger(const string& strVal, int nRow, int nCol)
    : CCellDescrBase(strVal, nRow, nCol) {}

CCellDescrInteger::CCellDescrInteger(int nVal, int nRow, int nCol)
    : CCellDescrBase(to_string(nVal), nRow, nCol) {}

CCellDescrInteger::CCellDescrInteger(const CCellDescrInteger& cell)
    : CCellDescrBase(cell) {}
```

ПРЕОБРАЗУВАНЕТО НИЗ → ЧИСЛО

```
int CCellDescrInteger::getInt() const
{
    try
    {
        size_t nPos = 0;
```

```

        int nVal = stoi(m_str_val, &nPos);

        // stoi спира при първия негоден символ. Ако е останало нещо
        // освен празни знаци, стойността не е цяло число.
        while (nPos < m_str_val.length() && isspace(static_cast<unsigned char>
(m_str_val[nPos])))
            nPos++;
        if (nPos != m_str_val.length())
            throw CCellException("Клетка " + getAddress() + ": \"" + m_str_val +
                                "\" не е цяло число");

        return nVal;
    }
    catch (const invalid_argument&)
    {
        throw CCellException("Клетка " + getAddress() + ": \"" + m_str_val +
                                "\" не е цяло число");
    }
    catch (const out_of_range&)
    {
        throw CCellException("Клетка " + getAddress() + ": \"" + m_str_val +
                                "\" е извън обхвата на int");
    }
}

```

ТРИТЕ СЛУЧАЯ, КОИТО `stoi` РАЗЛИЧАВА

Вход	Какво прави <code>stoi</code>	Нашата реакция
"42"	връща 42, <code>nPos</code> = 2 = дължината	всичко е наред
"не-число"	хвърля <code>invalid_argument</code>	улавяме и хвърляме своето
"99999999999999999999"	хвърля <code>out_of_range</code>	улавяме и хвърляме своето
"42abc"	връща 42, <code>nPos</code> = 2 ≠ дължината — не хвърля нищо	проверката с <code>nPos</code> го улавя

Последният ред е капанът. `stoi("42abc")` връща 42 съвсем спокойно — чете колкото може и спира. Без проверката с `nPos` клетка със стойност "42abc" ще мине за валидно число. Затова вторият параметър на `stoi` не е излишен: той казва *докъде* е стигнало четенето.

Улавяме чуждо изключение и хвърляме свое. Това е обичайна практика: `invalid_argument` идва от стандартната библиотека и не знае нищо за клетки. Превръщаме го в `CCellException` със съобщение, което казва *коя* клетка и *каква* стойност — така грешката е полезна за потребителя, а не само за програмиста.

ИЗВЕЖДАНЕТО — ЧИСЛАТА ВДЯСНО

```

ostream& CCellDescrInteger::Output(ostream& toStream) const
{
    toStream << "[" << setw(3) << right << getAddress() << "]" "
              << setw(10) << right;

    try
    {

```

```

        toStream << getInt();                // числото, подравнено вдясно
    }
    catch (const CCellException&)
    {
        toStream << "#ЧИСЛО!";                // невалидна стойност в числова клетка
    }
    return toStream;
}

```

Защо `Output` улавя изключението вместо да го пусне нагоре? Защото извеждането на таблица не бива да спира заради една повредена клетка. Показваме `#ЧИСЛО!` — точно както прави Excel — и продължаваме. Това е съзнателно решение: изключението се улавя там, където има смислен начин да се продължи.

6 Операторите на числовата клетка

```

CCellDescrInteger& CCellDescrInteger::operator=(const CCellDescrInteger& cell)
{
    if (this == &cell) return *this;
    m_str_val = cell.m_str_val;
    m_row     = cell.m_row;
    m_col     = cell.m_col;
    return *this;
}

bool CCellDescrInteger::operator==(const CCellDescrInteger& cell) const
{
    return getInt() == cell.getInt();        // сравнява ЧИСЛАТА
}

bool CCellDescrInteger::operator<(const CCellDescrInteger& cell) const
{
    return getInt() < cell.getInt();
}

```

Сравнява се стойността, не позицията. Две клетки с еднакво число на различни места са „равни“ по този оператор. Това следва условието — сравнението е по данните.

Обърнете внимание, че операторите сравняват `getInt()`, а не `m_str_val`. Затова `"7"` и `"007"` биха били равни като числа. Точно за това е нужно упражнението: **как се пази** стойността и **как се тълкува** са два различни въпроса.

Тези оператори могат да хвърлят изключение. `getInt()` хвърля при невалидна стойност, значи и `==`, и `<` хвърлят. При сравняване на клетки с непроверени данни сложете сравнението в `try`.

7 CCellDescrString

Огледален на числовия клас, но по-прост — стойността вече е низ и не се преобразува нищо.

```

string CCellDescrString::getString() const { return m_str_val; }

bool CCellDescrString::operator==(const CCellDescrString& cell) const
{
    return m_str_val == cell.m_str_val;        // сравнява НИЗОВЕТЕ
}

bool CCellDescrString::operator<(const CCellDescrString& cell) const
{
    return m_str_val < cell.m_str_val;         // лексикографски
}

```

ИЗВЕЖДАНЕТО – ТЕКСТЪТ ВЛЯВО И В КАВИЧКИ

```

ostream& CCellDescrString::Output(ostream& toStream) const
{
    // padRight вместо setw: брои букви, не байтове (виж utf8Length)
    toStream << "[" << setw(3) << right << getAddress() << "]" "
        << padRight("\'" + m_str_val + "\'", 10);
    return toStream;
}

```

Защо не просто `setw(10) << left` ? Защото `setw` брои **байтове**, а кирилската буква в UTF-8 заема два. За "Иван" това са 8 байта плюс двете кавички — точно 10, тоест `setw(10)` не добавя нито един интервал и колоната се разминава.

Затова добавяме две помощни функции над класовете:

```

static size_t utf8Length(const string& str)
{
    size_t nLen = 0;
    for (size_t i = 0; i < str.length(); i++)
        if ((static_cast<unsigned char>(str[i]) & 0xC0) != 0x80)
            nLen++;
    return nLen;
}

static string padRight(const string& str, size_t nWidth)
{
    size_t nLen = utf8Length(str);
    if (nLen >= nWidth) return str;
    return str + string(nWidth - nLen, ' ');
}

```

В UTF-8 всеки знак започва с байт, при който горните два бита **не** са `10` ; продълженията винаги започват с `10` . Броим само началните — така получаваме броя букви.

ДВАТА ВИДА СРАВНЕНИЕ ВЪРХУ ЕДНИ И СЪЩИ ДАННИ

Това е най-показателният тест в упражнението:

```

като ЧИСЛА:  9 < 10 = да
като НИЗОВЕ: "9" < "10" = не

```

Като числа девет е по-малко от десет. Като низове сравнението е буква по буква: `'9'` има код 57, `'1'` — код 49, значи `"9"` е след `"10"`. Един и същ текст, две различни подредби — точно затова са нужни два различни класа.

8 Шаблонният клас `CLess`

Условието иска шаблонен клас за сравнение с оператор-функция.

```
template <class T>
class CLess
{
public:
    // Оператор-функция: прави обекта извикваем като функция.
    bool operator()(const T& left, const T& right) const
    {
        return left < right;
    }
};
```

КАКВО Е ОПЕРАТОР `()`

Предефинирането на `operator()` прави обекта **извикваем** — може да се използва със скоби, като функция:

```
CLess<int> lessInt;
lessInt(3, 8);           // вика lessInt.operator()(3, 8), връща true
```

Такъв обект се нарича **функционален обект** или *функтор*. За разлика от обикновена функция, той може да носи и състояние.

Как работи `template <class T>`. Класът не съществува сам по себе си. При `CLess<int>` компилаторът генерира отделен клас, в който `T` е заменено с `int`; при `CLess<string>` — още един. Затова грешките в шаблони се появяват чак когато шаблонът се използва.

Единственото изискване към `T` е да има `operator<`. Нашите `CCellDescrInteger` и `CCellDescrString` го имат — затова `CLess<CCellDescrInteger>` работи, без да е писано нищо специално.

ШАБЛОННА ФУНКЦИЯ, КОЯТО ПОЛЗВА ШАБЛОННИЯ КЛАС

```
template <class T>
const T& Smaller(const T& left, const T& right)
{
    CLess<T> less;
    return less(left, right) ? left : right;
}
```

Тук типът дори не се пише при извикване — `Smaller(15, 4)` сам разбира, че `T` е `int`.

Шаблоните се пишат изцяло в заглавния файл (тук — преди употребата им). Компилаторът трябва да види тялото, за да генерира версия за конкретния тип. Ако разделите обявяване и дефиниция както при обикновените класове, ще получите `LNK2019: unresolved external symbol` при свързването.

9 Главна функция

9.1 Масив от указатели към базовия клас

```
const int nCells = 6;
CCellDescrBase* arrCells[nCells];           // равен брой от двата вида

arrCells[0] = new CCellDescrInteger(42, 0, 0);    // A1
arrCells[1] = new CCellDescrString("Иван", 0, 1); // B1
arrCells[2] = new CCellDescrInteger(-7, 1, 0);    // A2
arrCells[3] = new CCellDescrString("Варна", 1, 1); // B2
arrCells[4] = new CCellDescrInteger(1000, 2, 0);  // A3
arrCells[5] = new CCellDescrString("00П", 2, 1);  // B3
```

Защо масивът е от указатели, а не от обекти? Три причини, всяка достатъчна сама по себе си:

1. CCellDescrBase е абстрактен — масив от такива обекти е невъзможен.
2. Дори да не беше, елементите са с различни размери и различни типове.
3. Полиморфизмът работи само през указател или референция. При масив от обекти наследената част би се отрязала.

9.2 Извеждане — всеки вид по своя начин

```
for (int i = 0; i < nCells; i++)
    cout << " " << *arrCells[i] << "  <- "
        << (i % 2 == 0 ? "целочислена" : "стрингова") << endl;
```

`*arrCells[i]` — първо `[]`, после `*`: взема се `i`-тият указател и се разадресира. В цикъла няма нито едно `if` за типа и въпреки това числата излизат вдясно, а текстът — вляво.

9.3 Четирите вида изключения

```
// а) невалидна стойност в целочислена клетка
CCellDescrInteger cellBad("не-число", 3, 0);
try
{
    int nVal = cellBad.getInt();
    cout << "  стойност: " << nVal << endl;
}
catch (const CCellException& ex)
{
    cout << "  хванато изключение: " << ex.what() << endl;
}
```

Забележете реда вътре в `try`. Първо `int nVal = cellBad.getInt();`, чак после извеждането. Ако напишете всичко на един ред:

```
cout << " стойност: " << cellHuge.getInt() << endl;    // не правете така
```

текстът `" стойност: "` вече е отишъл на екрана, когато изключението бъде хвърлено. Редът излиза разкъсан и съобщението за грешка се залепва за него. Правилото: **в `try` първо се върши работата, после се извежда.**

```
// в) невалиден ред – изключение ОТ КОНСТРУКТОРА
try
{
    CCellDescrInteger cellErr(1, -5, 0);
    cout << " създадена клетка " << cellErr.getAddress() << endl;
}
catch (const CCellException& ex)
{
    cout << " хванато изключение: " << ex.what() << endl;
}
```

Изключение от конструктор. Обектът `cellErr` никога не се създава — вторият ред не се изпълнява. Това е и причината проверките да са точно в конструктора: невалиден обект просто не може да съществува.

```
// д) невалиден индекс в масива
const int arrIdx[] = { 2, 99, 4 };
for (int k = 0; k < 3; k++)
{
    int i = arrIdx[k];
    try
    {
        if (i < 0 || i >= nCells)
            throw CCellException("Невалиден индекс " + to_string(i) +
                                " (допустимо 0.." + to_string(nCells - 1) + ")");
        cout << " arrCells[" << i << "] = " << *arrCells[i] << endl;
    }
    catch (const CCellException& ex)
    {
        cout << " хванато изключение: " << ex.what() << endl;
    }
}
```

`try` е **вътре** в цикъла. Така грешката при един индекс не прекъсва обхождането — след улавянето цикълът продължава със следващия. Ако `try` обграждаше целия цикъл, първата грешка щеше да го прекрати.

`catch` винаги приема по референция: `catch (const CCellException& ex)`. При `catch (CCellException ex)` изключението се копира излишно, а при йерархия от изключения копието „отрязва“ наследника до базовия клас и съобщението се губи.

9.4 Освобождаване на паметта

```
for (int i = 0; i < nCells; i++)
{
    delete arrCells[i];          // вика правилния деструктор – той е virtual
    arrCells[i] = nullptr;
}
```

Всяко `new` в 9.1 се вдвоява с `delete` тук. Деструкторът на базовия клас е `virtual`, затова се вика деструкторът на истинския тип.

Компилиране и стартиране

Клавиши	Действие
<code>Ctrl</code> + <code>Shift</code> + <code>B</code>	Build.
<code>F5</code>	Start Debugging. Дебъгерът спира при неуловено изключение и показва къде е хвърлено.
<code>Ctrl</code> + <code>Alt</code> + <code>E</code>	Exception Settings — може да се нареди дебъгерът да спира и при <i>уловените</i> изключения. Полезно при търсене на грешка.

```
cl /EHsc /std:c++17 /utf-8 cells.cpp
cells.exe
```

Флагът `/EHsc` тук е задължителен, а не препоръчителен. Без него Visual Studio не генерира код за обработка на изключения и `try/catch` не работи както трябва. Предупреждението е `C4530: C++ exception handler used, but unwind semantics are not enabled`. В средата на Visual Studio флагът е включен по подразбиране.

Кирилица под Windows

Файлът се записва като *Unicode (UTF-8 with signature)* — *Codepage 65001*; компилаторът получава `/utf-8`; конзолата се превключва с `chcp 65001` или със `system("chcp 65001 > nul");` като първи ред в `main` (нужен е `#include <cstdlib>`).

Помощните функции `utf8Length` и `padRight` от стъпка 7 решават *подравняването*. Те не заместват горните три настройки — те определят дали буквите изобщо ще се покажат правилно.

Пълен код на програмата

Ако някъде сте се загубили, сравнете с този листинг:

```
// Тема 6: Абстрактни класове, типови преобразувания, изключения,
//          оператори и шаблони.
// Описател на клетка на таблица – базов абстрактен клас и два наследника.

#include <cstdio>      // getchar
#include <exception>
#include <iomanip>      // setw, left, right
#include <iostream>
#include <stdexcept>    // invalid_argument, out_of_range
#include <string>
using namespace std;

// =====
// Собствен клас за изключения
// =====
class CCellException : public exception
{
private:
    string m_strMsg;

public:
    explicit CCellException(const string& strMsg) : m_strMsg(strMsg) {}

    // what() е виртуална в std::exception – подменяме я.
    // noexcept е задължително: базовата версия обещава да не хвърля.
    virtual const char* what() const noexcept { return m_strMsg.c_str(); }
};

// =====
// Помощна функция: подравняване при кирилица
// =====
// setw брои БАЙТОВЕ, а кирилската буква в UTF-8 е 2 байта. Затова
// setw(10) върху "Иван" не добавя нищо – низът вече е 8 байта.
// Тази функция брои БУКВИ: началните байтове на UTF-8 знак са тези,
// при които горните два бита НЕ са 10.
static size_t utf8Length(const string& str)
{
    size_t nLen = 0;
    for (size_t i = 0; i < str.length(); i++)
        if ((static_cast<unsigned char>(str[i]) & 0xC0) != 0x80)
            nLen++;
    return nLen;
}

static string padRight(const string& str, size_t nWidth)
{
    size_t nLen = utf8Length(str);
    if (nLen >= nWidth) return str;
    return str + string(nWidth - nLen, ' ');
}

// =====
```

```

// I. Абстрактен клас CCellDescrBase
// =====
class CCellDescrBase
{
// Скрити член променливи. protected, за да ги достигат наследниците.
protected:
    string m_str_val;    // стойност на клетката, представя цяло число или низ
    int     m_row;       // ред
    int     m_col;       // колона

public:
    // Границите на таблицата – статични, общи за всички клетки
    static const int MAX_ROW = 100;
    static const int MAX_COL = 26;

    // Конструктори
    CCellDescrBase();                // подразбиращ се
    CCellDescrBase(const string& strVal, int nRow, int nCol); // експлицитен
    CCellDescrBase(const CCellDescrBase& cell);             // копиращ

    virtual ~CCellDescrBase();                // виртуален!

    // Акцесори / мутатори
    int  getRow() const;
    int  getCol() const;
    void setRow(int nRow);    // хвърля CCellException при невалиден ред
    void setCol(int nCol);    // хвърля CCellException при невалидна колона

    const string& getValue() const;
    void setValue(const string& strVal);

    // Адрес на клетката в човешки вид: A1, B3, ...
    string getAddress() const;

    // Напълно виртуална функция за извеждане в поток
    virtual ostream& Output(ostream& toStream) const = 0;
};

const int CCellDescrBase::MAX_ROW;
const int CCellDescrBase::MAX_COL;

CCellDescrBase::CCellDescrBase()
    : m_str_val(""), m_row(0), m_col(0) {}

CCellDescrBase::CCellDescrBase(const string& strVal, int nRow, int nCol)
    : m_str_val(strVal), m_row(0), m_col(0)
{
    setRow(nRow);    // през мутаторите, за да мине проверката
    setCol(nCol);
}

CCellDescrBase::CCellDescrBase(const CCellDescrBase& cell)
    : m_str_val(cell.m_str_val), m_row(cell.m_row), m_col(cell.m_col) {}

CCellDescrBase::~CCellDescrBase() {}

int CCellDescrBase::getRow() const { return m_row; }
int CCellDescrBase::getCol() const { return m_col; }

```

```

void CCellDescrBase::setRow(int nRow)
{
    if (nRow < 0 || nRow >= MAX_ROW)
        throw CCellException("Невалиден ред: " + to_string(nRow) +
                               " (допустимо 0.." + to_string(MAX_ROW - 1) + ")");
    m_row = nRow;
}

void CCellDescrBase::setCol(int nCol)
{
    if (nCol < 0 || nCol >= MAX_COL)
        throw CCellException("Невалидна колона: " + to_string(nCol) +
                               " (допустимо 0.." + to_string(MAX_COL - 1) + ")");
    m_col = nCol;
}

const string& CCellDescrBase::getValue() const { return m_str_val; }
void CCellDescrBase::setValue(const string& strVal) { m_str_val = strVal; }

string CCellDescrBase::getAddress() const
{
    string strAddr;
    strAddr += static_cast<char>('A' + m_col);
    strAddr += to_string(m_row + 1);
    return strAddr;
}

// ----- Един оператор за цялата йерархия -----
// Делегира на виртуалната Output – затова извежда според ТИПА НА ОБЕКТА.
ostream& operator<<(ostream& toStream, const CCellDescrBase& cell)
{
    return cell.Output(toStream);
}

// =====
// II. CCellDescrInteger – клетка с целочислени данни
// =====
class CCellDescrInteger : public CCellDescrBase
{
public:
    CCellDescrInteger();
    CCellDescrInteger(const string& strVal, int nRow, int nCol);
    CCellDescrInteger(int nVal, int nRow, int nCol); // удобен вариант
    CCellDescrInteger(const CCellDescrInteger& cell);
    virtual ~CCellDescrInteger();

    // Четене на данните като цяло число; хвърля при невалидна стойност
    int getInt() const;

    // Оператор за присвояване
    CCellDescrInteger& operator=(const CCellDescrInteger& cell);

    // Сравнения по ЦЕЛОЧИСЛЕНАТА стойност
    bool operator==(const CCellDescrInteger& cell) const;
    bool operator< (const CCellDescrInteger& cell) const;

    // Извеждане: числата се подравняват ВДЯСНО
    virtual ostream& Output(ostream& toStream) const;
};

```

```

CCellDescrInteger::CCellDescrInteger() : CCellDescrBase("0", 0, 0) {}

CCellDescrInteger::CCellDescrInteger(const string& strVal, int nRow, int nCol)
    : CCellDescrBase(strVal, nRow, nCol) {}

CCellDescrInteger::CCellDescrInteger(int nVal, int nRow, int nCol)
    : CCellDescrBase(to_string(nVal), nRow, nCol) {}

CCellDescrInteger::CCellDescrInteger(const CCellDescrInteger& cell)
    : CCellDescrBase(cell) {}

CCellDescrInteger::~~CCellDescrInteger() {}

// ----- Типовото преобразуване низ -> цяло число -----
int CCellDescrInteger::getInt() const
{
    try
    {
        size_t nPos = 0;
        int nVal = stoi(m_str_val, &nPos);

        // stoi спира при първия негоден символ. Ако е останало нещо
        // освен празни знаци, стойността не е цяло число.
        while (nPos < m_str_val.length() && isspace(static_cast<unsigned char>
(m_str_val[nPos])))
            nPos++;
        if (nPos != m_str_val.length())
            throw CCellException("Клетка " + getAddress() + ": \"" + m_str_val +
                "\" не е цяло число");

        return nVal;
    }
    catch (const invalid_argument&)
    {
        throw CCellException("Клетка " + getAddress() + ": \"" + m_str_val +
            "\" не е цяло число");
    }
    catch (const out_of_range&)
    {
        throw CCellException("Клетка " + getAddress() + ": \"" + m_str_val +
            "\" е извън обхвата на int");
    }
}

CCellDescrInteger& CCellDescrInteger::operator=(const CCellDescrInteger& cell)
{
    if (this == &cell) return *this;
    m_str_val = cell.m_str_val;
    m_row     = cell.m_row;
    m_col     = cell.m_col;
    return *this;
}

bool CCellDescrInteger::operator==(const CCellDescrInteger& cell) const
{
    return getInt() == cell.getInt();          // сравнява ЧИСЛАТА
}

bool CCellDescrInteger::operator<(const CCellDescrInteger& cell) const

```

```

{
    return getInt() < cell.getInt();
}

ostream& CCellDescrInteger::Output(ostream& toStream) const
{
    toStream << "[" << setw(3) << right << getAddress() << "]" "
        << setw(10) << right;
    try
    {
        toStream << getInt();                // числото, подравнено вдясно
    }
    catch (const CCellException&)
    {
        toStream << "#ЧИСЛО!";                // невалидна стойност в числова клетка
    }
    return toStream;
}

// =====
// III. CCellDescrString – клетка със стрингови данни
// =====
class CCellDescrString : public CCellDescrBase
{
public:
    CCellDescrString();
    CCellDescrString(const string& strVal, int nRow, int nCol);
    CCellDescrString(const CCellDescrString& cell);
    virtual ~CCellDescrString();

    // Четене на данните като низ
    string getString() const;

    CCellDescrString& operator=(const CCellDescrString& cell);

    // Сравнения по СТРИНГОВАТА стойност
    bool operator==(const CCellDescrString& cell) const;
    bool operator< (const CCellDescrString& cell) const;

    // Извеждане: текстът се подравнява ВЛЯВО и се огражда с кавички
    virtual ostream& Output(ostream& toStream) const;
};

CCellDescrString::CCellDescrString() : CCellDescrBase("", 0, 0) {}

CCellDescrString::CCellDescrString(const string& strVal, int nRow, int nCol)
    : CCellDescrBase(strVal, nRow, nCol) {}

CCellDescrString::CCellDescrString(const CCellDescrString& cell)
    : CCellDescrBase(cell) {}

CCellDescrString::~~CCellDescrString() {}

string CCellDescrString::getString() const { return m_str_val; }

CCellDescrString& CCellDescrString::operator=(const CCellDescrString& cell)
{
    if (this == &cell) return *this;
    m_str_val = cell.m_str_val;
}

```

```

        m_row      = cell.m_row;
        m_col      = cell.m_col;
        return *this;
    }

bool CCellDescrString::operator==(const CCellDescrString& cell) const
{
    return m_str_val == cell.m_str_val;          // сравнява НИЗОВЕТЕ
}

bool CCellDescrString::operator<(const CCellDescrString& cell) const
{
    return m_str_val < cell.m_str_val;           // лексикографски
}

ostream& CCellDescrString::Output(ostream& toStream) const
{
    // padRight вместо setw: брой букви, не байтове (виж utf8Length)
    toStream << "[" << setw(3) << right << getAddress() << "]" "
        << padRight("\\" + m_str_val + "\\", 10);
    return toStream;
}

// =====
// Шаблонен клас за сравнение CLess
// =====
template <class T>
class CLess
{
public:
    // Оператор-функция: прави обекта извикваем като функция.
    bool operator()(const T& left, const T& right) const
    {
        return left < right;
    }
};

// Помощна шаблонна функция – връща по-малкия от двата, чрез CLess
template <class T>
const T& Smaller(const T& left, const T& right)
{
    CLess<T> less;
    return less(left, right) ? left : right;
}

// =====
// IV. Главна функция
// =====
int main()
{
    // ===== 1-2. Масив от указатели към БАЗОВИЯ клас =====
    cout << "===== 1-2. Масив от указатели към CCellDescrBase =====" << endl;

    const int nCells = 6;
    CCellDescrBase* arrCells[nCells];          // равен брой от двата вида

    arrCells[0] = new CCellDescrInteger(42, 0, 0);          // A1
    arrCells[1] = new CCellDescrString("Иван", 0, 1);       // B1
    arrCells[2] = new CCellDescrInteger(-7, 1, 0);          // A2

```

```

arrCells[3] = new CCellDescrString("Варна", 1, 1);        // B2
arrCells[4] = new CCellDescrInteger(1000, 2, 0);          // A3
arrCells[5] = new CCellDescrString("00П", 2, 1);          // B3

// ===== 3. Извеждане чрез операторите – всеки вид по своя начин =====
cout << "---- Числата вдясно, текстът вляво ----" << endl;
for (int i = 0; i < nCells; i++)
    cout << " " << *arrCells[i] << "    <- "
        << (i % 2 == 0 ? "целочислена" : "стрингова") << endl;

// ===== 4. Обработка на изключения =====
cout << endl << "===== 4. Изключения =====" << endl;

// а) невалидна стойност в целочислена клетка
cout << "---- Невалидна стойност ----" << endl;
CCellDescrInteger cellBad("не-число", 3, 0);
try
{
    int nVal = cellBad.getInt();
    cout << "    стойност: " << nVal << endl;
}
catch (const CCellException& ex)
{
    cout << "    хванато изключение: " << ex.what() << endl;
}

// б) стойност извън обхвата на int
cout << "---- Извън обхвата на int ----" << endl;
CCellDescrInteger cellHuge("9999999999999999999", 3, 1);
try
{
    // Извикваме ПРЕДИ извеждането: иначе "стойност: " вече е на екрана,
    // когато изключението бъде хвърлено, и редът излиза разкъсан.
    int nHuge = cellHuge.getInt();
    cout << "    стойност: " << nHuge << endl;
}
catch (const CCellException& ex)
{
    cout << "    хванато изключение: " << ex.what() << endl;
}

// в) невалиден ред
cout << "---- Невалиден ред ----" << endl;
try
{
    CCellDescrInteger cellErr(1, -5, 0);
    cout << "    създадена клетка " << cellErr.getAddress() << endl;
}
catch (const CCellException& ex)
{
    cout << "    хванато изключение: " << ex.what() << endl;
}

// г) невалидна колона
cout << "---- Невалидна колона ----" << endl;
try
{
    CCellDescrString cellErr("текст", 0, 99);
    cout << "    създадена клетка " << cellErr.getAddress() << endl;
}

```

```

}
catch (const CCellException& ex)
{
    cout << "    хванато изключение: " << ex.what() << endl;
}

// д) невалиден индекс в масива
cout << "---- Невалиден индекс в масива ----" << endl;
const int arrIdx[] = { 2, 99, 4 };
for (int k = 0; k < 3; k++)
{
    int i = arrIdx[k];
    try
    {
        if (i < 0 || i >= nCells)
            throw CCellException("Невалиден индекс " + to_string(i) +
                                   " (допустимо 0.." + to_string(nCells - 1) + ")");
        cout << "    arrCells[" << i << "] = " << *arrCells[i] << endl;
    }
    catch (const CCellException& ex)
    {
        cout << "    хванато изключение: " << ex.what() << endl;
    }
}

// е) клетката с невалидна стойност се извежда, без да събори програмата
cout << "---- Извеждане на повредена клетка ----" << endl;
cout << "    " << cellBad << "    <- Output хваща изключението вътре" << endl;

// ===== Оператори за сравнение =====
cout << endl << "===== Оператори == и < =====" << endl;
CCellDescrInteger nA(42, 0, 0), nB(42, 5, 5), nC(7, 1, 1);
cout << "    nA(42) == nB(42) [различни позиции]: " << (nA == nB ? "да" : "не") << endl;
cout << "    nA(42) == nC(7): " << (nA == nC ? "да" : "не") << endl;
cout << "    nC(7) < nA(42): " << (nC < nA ? "да" : "не") << endl;

CCellDescrString sA("Ана", 0, 0), sB("Ана", 3, 3), sC("Борис", 1, 1);
cout << "    sA(Ана) == sB(Ана): " << (sA == sB ? "да" : "не") << endl;
cout << "    sA(Ана) < sC(Борис) [лексикограф.]: " << (sA < sC ? "да" : "не") << endl;

// Разликата между двата вида сравнение
cout << "---- Едни и същи данни, различно сравнение ----" << endl;
CCellDescrInteger n9("9", 0, 0), n10("10", 0, 1);
CCellDescrString s9("9", 0, 0), s10("10", 0, 1);
cout << "    като ЧИСЛА: 9 < 10 = " << (n9 < n10 ? "да" : "не") << endl;
cout << "    като НИЗОВЕ: \"9\" < \"10\" = " << (s9 < s10 ? "да" : "не")
    << "    <- '9' е след '1' в таблицата" << endl;

// ===== Оператор за присвояване =====
cout << endl << "===== Оператор = =====" << endl;
CCellDescrInteger nDst(0, 4, 4);
cout << "    преди: " << nDst << endl;
nDst = nA;
cout << "    след nDst = nA: " << nDst << endl;

// ===== Шаблонният клас CLess =====
cout << endl << "===== Шаблонен клас CLess =====" << endl;
CLess<int>    lessInt;
CLess<string> lessStr;

```

```

CLess<double> lessDbl;

cout << " CLess<int>(3, 8)          = " << (lessInt(3, 8) ? "да" : "не") << endl;
cout << " CLess<int>(8, 3)          = " << (lessInt(8, 3) ? "да" : "не") << endl;
cout << " CLess<string>(Ана, Борис) = " << (lessStr("Ана", "Борис") ? "да" : "не") <<
endl;
cout << " CLess<double>(2.5, 2.4)    = " << (lessDbl(2.5, 2.4) ? "да" : "не") << endl;

// Същият шаблон работи и върху нашите класове – те имат operator<
CLess<CCellDescrInteger> lessCellInt;
CLess<CCellDescrString> lessCellStr;
cout << " CLess<CCellDescrInteger>(nC(7), nA(42)) = "
    << (lessCellInt(nC, nA) ? "да" : "не") << endl;
cout << " CLess<CCellDescrString>(sA, sC)          = "
    << (lessCellStr(sA, sC) ? "да" : "не") << endl;

cout << "---- Шаблонна функция, която ползва CLess ----" << endl;
cout << " Smaller(15, 4)          = " << Smaller(15, 4) << endl;
cout << " Smaller(string) на 2 низа= " << Smaller(string("круша"), string("ябълка")) <<
endl;
cout << " Smaller(nC, nA)          = " << Smaller(nC, nA) << endl;

// Сортиране на масив от клетки чрез CLess
cout << "---- Сортиране на числови клетки чрез CLess ----" << endl;
CCellDescrInteger arrSort[5] = {
    CCellDescrInteger(50, 0, 0), CCellDescrInteger(-3, 1, 0),
    CCellDescrInteger(17, 2, 0), CCellDescrInteger(0, 3, 0),
    CCellDescrInteger(99, 4, 0)
};
CLess<CCellDescrInteger> comp;
for (int i = 0; i < 4; i++)
    for (int j = 0; j < 4 - i; j++)
        if (comp(arrSort[j + 1], arrSort[j])) // дясната е по-малка
        {
            CCellDescrInteger tmp = arrSort[j];
            arrSort[j] = arrSort[j + 1];
            arrSort[j + 1] = tmp;
        }
for (int i = 0; i < 5; i++)
    cout << " " << arrSort[i] << endl;

// ===== Освобождаване на паметта =====
cout << endl << "===== Освобождаване =====" << endl;
for (int i = 0; i < nCells; i++)
{
    delete arrCells[i]; // вика правилния деструктор – той е virtual
    arrCells[i] = nullptr;
}
cout << " всички " << nCells << " клетки са освободени" << endl;

// Задържа конзолния прозорец отворен при стартиране с F5 (debug)
getchar();
return 0;
}

```

Очакван изход

```
===== 1-2. Масив от указатели към CCellDescrBase =====
--- Числата вдясно, текстът вляво ---
[ A1]      42    <- целочислена
[ B1] "Иван"     <- стрингова
[ A2]     -7    <- целочислена
[ B2] "Варна"   <- стрингова
[ A3]    1000   <- целочислена
[ B3] "ООП"     <- стрингова

===== 4. Изключения =====
--- Невалидна стойност ---
    хванато изключение: Клетка A4: "не-число" не е цяло число
--- Извън обхвата на int ---
    хванато изключение: Клетка B4: "99999999999999999999" е извън обхвата на int
--- Невалиден ред ---
    хванато изключение: Невалиден ред: -5 (допустимо 0..99)
--- Невалидна колона ---
    хванато изключение: Невалидна колона: 99 (допустимо 0..25)
--- Невалиден индекс в масива ---
    arrCells[2] = [ A2]      -7
    хванато изключение: Невалиден индекс 99 (допустимо 0..5)
    arrCells[4] = [ A3]    1000
--- Извеждане на повредена клетка ---
[ A4] #ЧИСЛО!    <- Output хваща изключението вътре

===== Оператори == и < =====
nA(42) == nB(42) [различни позиции]: да
nA(42) == nC(7):                      не
nC(7)  <  nA(42):                      да
sA(Ана) == sB(Ана):                   да
sA(Ана) <  sC(Борис) [лексикограф.]: да
--- Едни и същи данни, различно сравнение ---
    като ЧИСЛА:  9 < 10 = да
    като НИЗОВЕ: "9" < "10" = не    <- '9' е след '1' в таблицата

===== Оператор = =====
    преди: [ E5]      0
    след nDst = nA: [ A1]      42

===== Шаблонен клас CLess =====
    CLess<int>(3, 8)          = да
    CLess<int>(8, 3)          = не
    CLess<string>(Ана, Борис) = да
    CLess<double>(2.5, 2.4)   = не
    CLess<CCellDescrInteger>(nC(7), nA(42)) = да
    CLess<CCellDescrString>(sA, sC)          = да
--- Шаблонна функция, която ползва CLess ---
    Smaller(15, 4)            = 4
    Smaller(string) на 2 низа = круша
    Smaller(nC, nA)           = [ B2]      7
--- Сортиране на числови клетки чрез CLess ---
[ A2]      -3
[ A4]       0
[ A3]      17
[ A1]      50
[ A5]      99
```

===== Освобождаване =====
всички 6 клетки са освободени

Три реда, които си струва да се разгледат.

[A4] #ЧИСЛО! — клетката с невалидна стойност се извежда, без да събори програмата: `Output` улавя изключението вътре.

като `НИЗОВЕ: "9" < "10" = не` — лексикографското сравнение от стъпка 7.

`Smaller(nC, nA) = [ B2] 7` — шаблонната функция върна нашия клас и той се изведе с оператора `си`.
Шаблонът не знае нищо за клетки.

Чести грешки

Код	Съобщение	Причина и решение
C2259	cannot instantiate abstract class	Опит за <code>CCellDescrBase cell;</code> , или наследник не е реализирал <code>Output</code> .
C2694	overriding virtual function has less restrictive exception specification	Липсва <code>noexcept</code> при <code>what()</code> .
C4530	C++ exception handler used, but unwind semantics are not enabled	Липсва флаг <code>/EHsc</code> .
LNK2019	unresolved external symbol ... <code>CLess<...></code>	Тялото на шаблона е отделено от обявяването. Шаблоните се пишат цели на едно място.
C2676	binary '<': does not define this operator	<code>CLess<T></code> е използван с тип, който няма <code>operator<</code> .
—	<i>Unhandled exception</i> и програмата спира	Изключението е хвърлено извън <code>try</code> , или <code>catch</code> е за друг тип. Добавете <code>catch (const exception& ex)</code> като последен.
—	<code>"42abc"</code> минава за валидно число	Липсва проверката с <code>nPos</code> след <code>stoi</code> (стъпка 5).
—	Редът с грешката излиза разкъсан	Извеждането и извикването са на един ред в <code>try</code> (стъпка 9.3).
—	Колоните с текст не се подравняват	<code>setw</code> брой байтове. Използвайте <code>padRight</code> (стъпка 7).
—	Теч на памет	Липсва <code>delete</code> за някой елемент от масива, или деструкторът на базовия клас не е <code>virtual</code> .
—	Клетка с ред <code>-5</code> се създава успешно	Конструкторът записва направо в списъка за инициализация, вместо да вика <code>setRow</code> (стъпка 3).

Обобщение на темата

Абстрактен клас	Поне една функция с <code>= 0</code> . Обект не може да се създаде; указател и референция — може. Служи за общ интерфейс.
Типово преобразуване	<code>stoi</code> с втори параметър — само така се улавя <code>"42abc"</code> . Хвърля <code>invalid_argument</code> и <code>out_of_range</code> .
Изключения	<code>throw</code> прекъсва функцията; <code>catch</code> по референция; хващат се там, където има смислен начин да се продължи.
Шаблони	Един код за много типове. Единственото изискване към типа е да поддържа операциите, които шаблонът използва.
Функтор	Клас с <code>operator()</code> — обект, който се вика като функция. Основата на всичко в STL алгоритмите.

Какво следва

Упражнение 9	<code>std::vector</code> и алгоритми — контейнер вместо ръчния масив от упражнение 3.
Упражнение 10	Итератори и алгоритмите <code>sort</code> , <code>unique</code> , <code>remove</code> — те приемат точно такива функтори като <code>Cless</code> .
Упражнения 12–13	<code>map</code> и <code>set</code> — асоциативни контейнери, които вътрешно ползват сравнител, точно като <code>Cless</code> .